OFFLINE STORAGE SYSTEM

Automatic Integration Guide

Connect the Python Document Processor and Python Audio Processor to the central intranet storage server

Complete step-by-step implementation • code templates • verification • troubleshooting

Purpose
This guide explains how to automatically send each successfully processed document/audio input file and its generated output to the Offline Storage System. It is written so the integration can be completed without needing live assistance.

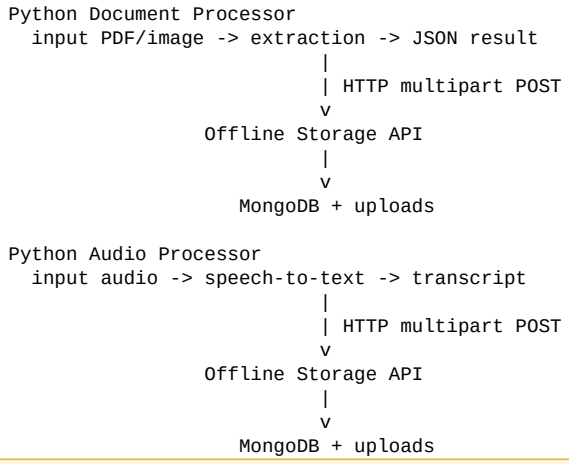
Important scope note: The central storage repository was inspected before preparing this guide. Its current routes accept multipart uploads at `POST /api/v1/records/documents` and `POST /api/v1/records/audio`. The integration code below is a Python template: the exact place to call it depends on each existing processing project's code and variable names. Do not paste it over your working processing logic.

Repository: https://github.com/dhanush-sr-5/Offline-Storage-System

Guide version: 1.0 | Prepared: 12 November 2026

1. What you are building

The two Python applications continue doing their existing work. After a processing job succeeds, each application sends the original input file plus its output to one central Offline Storage System server over the office intranet.



Design rule: The processor must not wait for a human to upload the same file again. Call the storage function automatically only after the processor has a successful result. If storage is temporarily unavailable, preserve the result locally and retry rather than losing it.

Responsibilities

• **Document project:** retain its existing PDF/image extraction logic; send original file and JSON output to the documents endpoint.

• **Audio project:** retain its existing audio/transcription logic; send original audio and transcript text to the audio endpoint.

• **Storage server:** validate the request, save the uploaded file, create a MongoDB record, and return a success response.

• **Dashboard:** read records from the central API; it does not need to run inside either Python processor.

2. Confirm the current storage API contract

The repository code currently defines these routes and field names. The upload middleware expects the multipart file field to be named exactly `inputFile`.

Purpose	HTTP endpoint	Required multipart fields
Document record	POST /api/v1/records/documents	inputFile, outputFormat=json, outputData=
Audio record	POST /api/v1/records/audio	inputFile, outputFormat=txt, outputData=
List records	GET /api/v1/records	No body
Get record output	GET /api/v1/records/{MongoDB record id}/output	No body
Health check	GET /api/health	No body

The storage controller currently requires: uploaded file; type (the route sets this automatically); outputFormat; outputData. sourceJobId is optional. For JSON, outputData is parsed with JSON.parse. For TXT, it is stored as text. The server generates its own processingId.

Important: Do not send the output in a field called `result`, `file`, or `transcript`. The current API expects the exact field names shown above.

3. Prepare the central storage server

1. Choose one always-on computer/server on the office intranet to host Offline Storage System. Give it a stable LAN IP address or DHCP reservation.

2. Install and start MongoDB on that server. Confirm the MongoDB service is running.

3. Start the Offline Storage backend using the deployment instructions in your repository. Confirm it reports MongoDB connected.

4. From the server itself, open `http://localhost:5000/api/health`. Expected response includes `"status":"success"` and `"database":"Connected"`.

5. From a different office PC, open `http://SERVER-LAN-IP:5000/api/health`. If it fails, check Windows Firewall and confirm the backend listens on `0.0.0.0`.

6. Record the server IP. In the examples below replace `192.168.1.50` with the actual IP.

```
# Example only - replace with your server's real IP
http://192.168.1.50:5000/api/health
```

Network/security note: allow inbound TCP 5000 only from the office subnet or the two processor machines. Do not expose this API to the public internet. The current repository code uses open CORS and does not yet show API-key authentication. Add authentication and network restrictions before handling sensitive/production data.

4. Prepare each Python processing project

4.1 Install the HTTP client

Open Command Prompt/PowerShell in each Python project's own folder. Activate its virtual environment first if it has one, then run:

```
python -m pip install requests
python -m pip show requests
```

If the project uses a requirements.txt file, add this line and install dependencies:

```
requests
# then:
python -m pip install -r requirements.txt
```

4.2 Create a configuration file

In each Python project create a file named `storage_config.py` (or use environment variables if the project already has configuration management).

```
# storage_config.py
STORAGE_BASE_URL = "http://192.168.1.50:5000"
REQUEST_TIMEOUT_SECONDS = 60
```

Use the real storage server IP. Do not use localhost from a different computer: localhost always means the computer running the Python process.

4.3 Optional environment-variable configuration

```
# Windows PowerShell (current terminal only)
$env:STORAGE_BASE_URL = "http://192.168.1.50:5000"

# Python
import os
STORAGE_BASE_URL = os.getenv(
    "STORAGE_BASE_URL",
    "http://192.168.1.50:5000"
)
```

5. Document project integration

Create a file such as `storage_client.py` in the document-processing project. This function sends the original document and extracted table data.

```
import json
import requests
from storage_config import STORAGE_BASE_URL, REQUEST_TIMEOUT_SECONDS
```

```
def send_document_to_storage(
    input_file_path: str,
    extracted_data,
    source_job_id: str
):
    url = f"{STORAGE_BASE_URL}/api/v1/records/documents"

    # Keep the file open until requests has finished sending it.
    with open(input_file_path, "rb") as input_file:
        response = requests.post(
            url,
            files={
                "inputFile": (
                    input_file_path.split("/")[-1],
                    input_file
                )
            },
            data={
                "outputFormat": "json",
                "outputData": json.dumps(
                    extracted_data,
                    ensure_ascii=False
                ),
                "sourceJobId": source_job_id
            },
            timeout=REQUEST_TIMEOUT_SECONDS
        )

    response.raise_for_status()
    return response.json()
```

Where to call it: Find the point in your document processor where extraction has completed and the final JSON object is available. Immediately after that successful step, call the function. Use the path to the original PDF/image, not a temporary preview image unless that is the intended source file.

```
# Example integration point inside your existing processing flow:
extracted_data = process_document(input_file_path)

# Only after process_document succeeds:
try:
    saved_record = send_document_to_storage(
        input_file_path=input_file_path,
        extracted_data=extracted_data,
        source_job_id=job_id
    )
    print("Saved in central storage:", saved_record)
except requests.RequestException as error:
    print("Central storage transfer failed:", error)
    # Preserve the local result and queue/retry it.
```

Replace process_document, input_file_path, extracted_data and job_id with the actual names used by your application. Do not call the storage function before extraction has completed.

6. Audio project integration

Create a similar `storage_client.py` in the audio project. The audio endpoint expects `outputFormat=txt` and the transcript as plain text.

```
import requests
from storage_config import STORAGE_BASE_URL, REQUEST_TIMEOUT_SECONDS

def send_audio_to_storage(
    audio_file_path: str,
    transcript: str,
    source_job_id: str
):
    url = f"{STORAGE_BASE_URL}/api/v1/records/audio"

    with open(audio_file_path, "rb") as audio_file:
        response = requests.post(
            url,
            files={
                "inputFile": (
                    audio_file_path.split("/")[-1],
                    audio_file
                )
            },
```

```
            data={
                "outputFormat": "txt",
                "outputData": transcript,
                "sourceJobId": source_job_id
            },
            timeout=REQUEST_TIMEOUT_SECONDS
        )

    response.raise_for_status()
    return response.json()
```

Where to call it

```
# Example integration point inside your existing audio flow:
transcript = transcribe_audio(audio_file_path)

# Only after transcription succeeds:
try:
    saved_record = send_audio_to_storage(
        audio_file_path=audio_file_path,
        transcript=transcript,
        source_job_id=job_id
    )
    print("Saved in central storage:", saved_record)
except requests.RequestException as error:
    print("Central storage transfer failed:", error)
    # Preserve the local transcript and queue/retry it.
```

Replace transcribe_audio, audio_file_path, transcript and job_id with your actual function/variable names. Keep your existing speech-to-text implementation unchanged.

7. Test the connection before changing production code

7.1 Test health endpoint from each processor PC

```
# Windows CMD or PowerShell
curl http://192.168.1.50:5000/api/health
```

Do not continue until this works from both Python project machines.

7.2 Test with a small sample document

```
import json
from storage_client import send_document_to_storage

sample_output = {
    "document_name": "integration-test.pdf",
    "rows": [
        {"item": "Sample", "quantity": 2}
    ]
}

result = send_document_to_storage(
    input_file_path="sample.pdf",
    extracted_data=sample_output,
    source_job_id="DOC-TEST-001"
)
print(json.dumps(result, indent=2))
```

7.3 Test with a small sample audio file

```
from storage_client import send_audio_to_storage

result = send_audio_to_storage(
    audio_file_path="sample.wav",
    transcript="This is an audio integration test.",
    source_job_id="AUDIO-TEST-001"
)
print(result)
```

7.4 Verify in the central storage system

```
# Check API health
curl http://192.168.1.50:5000/api/health

# List saved records
curl http://192.168.1.50:5000/api/v1/records
```

```
# Search by source job ID
curl "http://192.168.1.50:5000/api/v1/records/search?search=DOC-TEST-001"
```

Expected result: one document record with type=document and outputFormat=json; one audio record with type=audio and outputFormat=txt. Each should show the original filename, sourceJobId and saved output. The response's MongoDB record _id is used in the output retrieval endpoint.

8. Make it reliable: do not lose completed work

A direct HTTP call can fail if the storage server is rebooting, the network drops, or MongoDB is temporarily unavailable. A production-quality integration must not discard the completed output when this happens.

8.1 Minimum safe behavior

• Do not delete the processor's original input or output until the storage API confirms success.

• Log each transfer attempt with job ID, timestamp and error.

• Keep failed transfers in a local pending queue (for example, a JSONL file or SQLite database).

• Retry pending transfers with a delay (for example 5 seconds, 15 seconds, 60 seconds, then every 5 minutes).

• After success, mark the queue item as sent and retain the returned storage record ID.

• Do not create duplicate records on every retry. The current API does not enforce idempotency by sourceJobId; implement server-side duplicate protection before enabling automatic retries in production.

The following is a basic retry wrapper for a one-off test. It is not a durable queue and it does not prevent duplicates if the server saved the record but the response was lost.

```
import time
import requests

def send_with_basic_retry(send_function, attempts=4):
    delays = [5, 15, 60, 120]

    for attempt in range(attempts):
        try:
            return send_function()
        except requests.RequestException:
            if attempt == attempts - 1:
                raise
            time.sleep(delays[attempt])
```

For real unattended operation, use a durable local queue and add an idempotency key/unique sourceJobId rule to the storage backend. Do not treat the basic wrapper above as sufficient for production.

9. Troubleshooting

Connection refused / Failed to establish a new connection
Check that the storage backend is running; confirm its port is 5000; verify the server IP; test /api/health on the server and then from the processor PC. Ensure the backend binds to 0.0.0.0 and Windows Firewall allows TCP 5000 from the office subnet.

Request timed out
Check network connectivity and server load. Large audio/video files may take longer; increase REQUEST_TIMEOUT_SECONDS carefully. Confirm the server disk has enough free space.

404 Not Found
Check the exact path. Document endpoint is /api/v1/records/documents; audio endpoint is /api/v1/records/audio. Confirm there is no typo and the backend is mounted under /api/v1/records.

400: Please upload an input file
The multipart file field must be named inputFile. Use files={"inputFile": (...)} in requests.post.

400: Output format and output data are required
Send both outputFormat and outputData in the multipart data dictionary. Use json for documents and txt for audio.

400: Invalid JSON output data
The document output must be valid JSON. Use json.dumps(extracted_data), not str(extracted_data), and ensure the object contains only JSON-serializable values.

500: Failed to save processing record
Check backend terminal logs, MongoDB connectivity, disk permissions and upload directory availability. Verify MongoDB is connected at /api/health.

Health response says database Disconnected
Check MongoDB Windows service, MONGODB_URI in backend .env, and backend startup logs. Restart the backend after correcting configuration.

Works on server but not from another PC
Do not use localhost in the Python client. Use the storage server's LAN IP. Check firewall inbound rule, subnet/VLAN routing and that the server listens on 0.0.0.0.

Duplicate records after retry
The current API generates a new processingId for every successful POST and does not visibly enforce uniqueness on sourceJobId. Add an idempotency mechanism to the backend before enabling automatic retries.

Output saved but input file missing
Check upload folder permissions, disk capacity and that the upload middleware stores files in the configured uploads directory. Do not manually delete uploaded files.

Python says No module named requests
Activate the correct virtual environment and run python -m pip install requests using the same Python interpreter that runs the processor.

10. Security and office-network checklist

• Keep the storage API on the private office intranet; never port-forward it to the public internet.

• Restrict TCP 5000 inbound to the processor machines or trusted office subnet.

• Add API-key authentication (or stronger internal service authentication) to the storage backend before accepting sensitive files.

• Do not store API keys in source code or commit .env files. Use environment variables or protected configuration.

• Set upload size limits and allowed file types; scan/validate files according to office policy.

• Restrict MongoDB port 27017 to the storage server itself; processor PCs should call the HTTP API, not connect directly to MongoDB.

• Back up MongoDB and uploaded files together; test restore procedures.

• Use HTTPS with an internal certificate if data sensitivity or office policy requires it.

11. Common project layout

```
document_processor/
  main.py
  document_extractor.py
  storage_client.py
  storage_config.py
  requirements.txt
  logs/

audio_processor/
  main.py
  transcriber.py
  storage_client.py
  storage_config.py
  requirements.txt
  logs/

offline-storage-system/
  backend/
```

```
  frontend/
  deployment/
```

The filenames are suggestions. Keep the existing project layout if it is already organized differently.

12. Final acceptance checklist

■ Storage server has a stable LAN IP.

■ MongoDB service is running and /api/health reports Connected.

■ Health endpoint is reachable from the document processor PC.

■ Health endpoint is reachable from the audio processor PC.

■ requests is installed in both Python environments.

■ Document processor sends the original file and JSON only after successful extraction.

■ Audio processor sends the original file and transcript only after successful transcription.

■ Both records appear in GET /api/v1/records.

■ Document output can be retrieved and is valid JSON.

■ Audio output can be retrieved and is plain text.

■ A failed transfer preserves local input/output for later retry.

■ Authentication, firewall restrictions, duplicate protection and backups are addressed before production.

13. What this guide does not assume

The exact Python frameworks, processing function names, output object schemas, audio file extensions, document file extensions, and project folder structures were not provided. Therefore the code is deliberately framework-independent and uses placeholder names such as process_document(), transcribe_audio(), job_id and extracted_data. You must map those placeholders to the real variables/functions in each project.

The current storage API code inspected for this guide does not include API-key authentication or idempotency enforcement. It creates a fresh processingId per accepted upload. A reliable unattended deployment should add those controls before real production use.

The current server health endpoint reports API/database status; it does not prove that the dashboard UI, document processing integration, or audio processing integration has been fully validated. Test each acceptance item above on the actual office network.

14. Quick reference

```
Storage server:
  GET  http://SERVER-IP:5000/api/health
  GET  http://SERVER-IP:5000/api/v1/records
  POST http://SERVER-IP:5000/api/v1/records/documents
  POST http://SERVER-IP:5000/api/v1/records/audio

Document multipart fields:
  inputFile     = original PDF/image
  outputFormat  = json
  outputData    = json.dumps(extracted_data)
  sourceJobId   = stable job identifier (recommended)

Audio multipart fields:
  inputFile     = original audio file
  outputFormat  = txt
  outputData    = transcript string
  sourceJobId   = stable job identifier (recommended)
```

End of guide.